

Axiomatic Systems Programming: Structural Type Theory and Exact Arithmetic in C++23

The `dedekind` Authors

April 6, 2026

Abstract

This paper introduces `dedekind`, a C++23 library designed to embed formal mathematical structures directly within the language’s type system. We address the performance-productivity gap in computational science—often characterized as the “two-language problem”—by leveraging modern C++ features, including modules, concepts, and non-type template parameters (NTTP). Unlike traditional object-oriented frameworks that rely on nominal inheritance, `dedekind` employs a structural type theory to reify mathematical entities as compile-time specifications. We demonstrate the utility of this approach through a formal construction of the continuum $\mathbb{R}$ via Dedekind cuts, enabling the exact resolution of transcendental numbers without floating-point overhead. Furthermore, we show how hoisting logical invariants into type signatures allows the LLVM backend to perform aggressive structural pruning, such as collapsing contradictory set-builder predicates into zero-instruction machine code. This work illustrates that integrating formal mathematical rigor into systems programming can yield zero-overhead abstractions suitable for high-performance symbolic computing.

1 Introduction

The development of modern computational science, particularly in data science and Large Language Model (LLM) engineering, is constrained by a persistent performance-productivity gap known as the *two-language problem*. In this paradigm, researchers utilize high-level symbolic environments like `Python` for rapid prototyping and expressive reasoning, while relying on low-level languages like `C++` to handle execution backends. While tools such as `PyTorch` [1] and `NumPy` [2] offer elegant interfaces, they typically treat mathematical logic as a runtime implementation detail. This decoupling forces a manual translation layer between the high-level mathematical intent and the underlying hardware execution, leading to significant overhead in both developer time and system latency.

This duality is especially relevant in the context of LLM development and large-scale symbolic computing. In these fields, the ability to maintain the structural integrity of a mathematical model—such as the exactness of a tensor operation or the logical consistency of a set-builder predicate—is often lost during the transition to the `C++` backend. Traditional integration methods, such as `ctypes` or even modern binding tools like `nanobind` [3], bridge the binary gap but do not bridge the *semantic* gap. They allow `Python` to call `C++` functions, but they do not allow the `C++` compiler to reason about the high-level mathematical invariants defined in the symbolic layer.

Recent advances in systems programming have begun to challenge the historical divide between low-level performance and high-level abstraction. The `Rust` programming language, for instance, has gained significant traction by providing a “safety-first” approach through its affine type system and trait-based polymorphism. However, while `Rust` excels at memory safety and nominal constraints, its type system often operates as a “black box” that can obscure the

structural transparency required to hoist complex mathematical invariants into the heart of the optimization pipeline.

In contrast, the evolution of C++ toward the C++23 standard [4] has introduced a suite of features—notably `concepts`, `constexpr`, and non-type template parameters (NTP)—that enable a form of *structural typing*. Unlike traditional object-oriented hierarchies, these tools allow developers to define "mathematical species" based on their internal requirements and relational properties rather than their inherited names. As explored by Čukić [5], modern C++ is now sufficiently advanced to simulate, and in some cases surpass, the expressive power of pure functional programming languages. By treating types not merely as memory layouts but as compile-time predicates, C++ can now reify abstract "notions of computation" directly into the machine code.

This shift allows for the embedding of category-theoretic patterns, such as monads and functors, which serve as the formal foundation for structured side-effects and composition [6]. In the `dedekind` library, we leverage this functional substrate to treat mathematical entities as positions in a system of relations. Specifically, we utilize C++ `concepts` to enforce structural integrity at the call site, ensuring that the compiler acts as a verification engine. This approach transforms the C++ backend from a simple execution target into a sophisticated environment capable of reasoning about symbolic logic—effectively turning the "slow oracle" of a high-level symbolic system into a zero-overhead, statically-verified binary.

This shift toward compile-time verification mirrors a long-standing debate in the philosophy of mathematics between substance-based and structuralist ontologies. While traditional object-oriented programming relies on *nominal typing*—where an object's identity is tied to its named inheritance from a specific class—modern C++ features like `concepts` facilitate a *structural* approach. This aligns with the mathematical structuralism championed by the Bourbaki group and further formalized through category theory [7]. In this view, mathematical entities are not defined by what they "are" in isolation, but by the system of relations they inhabit.

The `dedekind` library specifically adopts the framework of *combinatorial species* introduced by Joyal [8, 9]. By treating mathematical structures as "species"—mappings that describe how a structure is placed upon a set of labels—we can reify abstract species (such as rings, fields, or even the continuum itself) as C++ template specifications. This "white-box" structural transparency allows the compiler to reason about the properties of a type (e.g., its commutativity or its completeness) purely by inspecting its requirements, much like McLarty's observation that "numbers can be just what they have to" within a categorical framework [10].

```
template <typename Universe, auto Predicate>
struct Set {
    using type = Universe;
    static constexpr auto p = Predicate;
};

// Mereological composition (Intersection)
template <typename S1, typename S2>
using Intersection = Set<typename S1::type, [](auto x) {
    return S1::p(x) && S2::p(x);
}>;
```

Listing 1: A simplified illustration of a set intersection implementation in C++ in the structuralist, mereological style.

In practice, this allows us to move beyond the "black-box" constraints of nominal traits. For instance, while a nominal system might require a type to explicitly `impl Real`, a structural system in C++ can identify a type as a representation of $\mathbb{R}$ if it satisfies the structural requirements of a Dedekind-complete ordered field. By hoisting these mereological invariants into the type signature, we enable the LLVM backend to perform aggressive structural pruning. As noted in the *n-Category Café* [11], this categorical perspective treats the "logic" of the system

and the "computation" of the system as dual aspects of the same structure—a principle we aim to exploit. For example, to ensure that contradictory set-builder predicates collapse into zero-instruction machine code (see Listing 1).

```
using
  = Set<
, [](auto x) { return x > 0.0; }>;
using
  = Set<
, [](auto x) { return false; }>;
// Intersection with
  is symbolically reduced
using Result = Intersection<
,
>;
// Proven at compile-time: Result is the empty set
static_assert(Result::p(5.0) == false);
```

Listing 2: A simplified illustration of the compiler simplifies the intersection with the empty set to a zero-cost identity.

The utility of this structuralist approach is best illustrated through two distinct computational advantages. First, the library enables aggressive structural pruning at compile-time. By reifying set operations as type-level compositions, the compiler can symbolically resolve the relationship between predicates. For instance, the intersection of two contradictory sets—such as the set of positive reals and the set of negative reals—can be identified as an empty set during translation to LLVM Intermediate Representation (IR). This allows the backend to prune entire execution branches, effectively collapsing complex symbolic logic into zero-instruction machine code.

```
// Representing e via the series predicate:
// q < sum_{n=0}^N 1/n! for some N
using e = Set<Q, [](Q q) {
  // Symbolic check:  $\exists N$  such that  $q < \text{partial\_sum}(N)$ 
  // In dedekind, this is resolved by the compiler's
  // ability to evaluate monotonic convergence.
  return  $\exists$ _approximation(q, [](int n) {
    return factorial_reciprocal_sum(n);
  });
}>;

// The compiler resolves this comparison at translation time
static_assert(e::p({271, 100}) == true); //  $2.71 < e$ 
```

Listing 3: Illustration of the symbolic construction of e as an intensional set (the lower cut $L \subset \mathbb{Q}$).

Furthermore, **dedekind** provides the ability to model intensional, transfinite sets through a purely symbolic approach. Rather than representing a collection as an extensional buffer of values, we define it by its membership condition. This is most powerfully demonstrated in our construction of the continuum $\mathbb{R}$ via Dedekind cuts. By representing a real number as a compile-time coordinate—a partition of the rational numbers $\mathbb{Q}$ —we enable the exact resolution of transcendental values without the rounding errors or overhead of standard floating-point arithmetic [12]. As shown in Listing 4, comparing e and π becomes a compile-time verification of their respective partitions.

```
// Defined as the lower cut of their respective series/properties
using e    = Set<Q, [](auto q) { return  $\exists$ _lower_bound(q, e_series); }>;
using  $\pi$     = Set<Q, [](auto q) { return  $\exists$ _lower_bound(q,  $\pi$ _series); }>;
```

```

/**
 * Structural comparison: Is  $e < \pi$ ?
 * This is true if there exists a rational 'r' such that:
 * r is in the lower cut of  $\pi$ , but NOT in the lower cut of  $e$ .
 */
static_assert( $e < \pi$ );

```

Listing 4: Illustration of the comparison of the transcendental constants e and π via their Dedekind partitions.

Together, these features bridge the gap between the declarative ease of symbolic algebra and the mechanical sympathy required for modern high-performance computing [13]. By allowing the C++ compiler itself to understand the symbolic logic of the Continuum, `dedekind` ensures that high-level mathematical invariants are preserved from specification to machine code, effectively treating the compiler as a formal proof assistant for systems-level performance.

2 Implementation

The methodology of `dedekind` is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next. By design, the flow of compilation mirrors a pedagogical progression: foundational, general-purpose modules are verified first, providing an immutable basis for the specialized and practical abstractions that follow.

2.1 Instrumented Model Validation: The CI Pipeline as an Independent Auditor

While formal verification in languages like Agda or Coq typically relies on static type-level proofs that are erased during compilation, the `dedekind` framework adopts a methodology of *Instrumented Model Validation*. By leveraging the LLVM profiling infrastructure, we provide an empirical *Runtime Witness* for our categorical primitives.

This approach serves as a deterministic evolution of the property-based testing tradition pioneered by QUICKCHECK [14]. Where QUICKCHECK utilizes stochastic search to falsify algebraic properties in Haskell, `dedekind` utilizes the CI/CD pipeline, which exercises `static_assert` (evaluated at compile-time) combined with `Catch2` tests (executed after unit compilation) to ensure that every structural identity is physically materialized in the binary. By enforcing high levels of patch coverage on new ontological partitions, we provide a "Physical Proof" in the form of *Binary Module Interface* (BMI) files that the abstract category theory—often treated as a ghost in the machine—is actually traversed by the hardware. This aligns with the "Real World" functional programming philosophy advocated by O’Sullivan et al. [15], where formal rigor is balanced with the practical necessity of observable, high-performance execution.

This methodology is physically enforced by a strictly layered directed acyclic graph (DAG) of the system’s build order. By carefully selecting C++ module partitions and build-step boundaries, we translate the dependencies of formal mathematics into a sequence of verified compilation units. This architecture mimics the pedagogical progression of a mathematical textbook, where the ontology matures from foundational axioms to complex theorems through a series of verified proofs. By isolating these prerequisites into a ‘category’ module, we leverage what Wadler termed "Theorems for Free!" [16]: once a structural identity is established and cached as a Binary Module Interface (BMI), its properties are inherited by all downstream partitions as an immutable "cached truth." This ensures that the compiler’s proof-search is grounded in a physical hierarchy, preventing circular reasoning and ensuring that the structural integrity of

the model is a prerequisite for successful linking. Crucially, the CI/CD methodology provides an independent, physical verification of this progression; by requiring every node in the build graph to pass its respective model validation before the pipeline can proceed, we ensure that the "textbook" derivation is not merely a logical possibility, but a materialized reality across the entire module graph.

2.1.1 Nominal vs. Structural Subtyping

Traditional object-oriented systems typically rely on *nominal subtyping*, where a type T is recognized as belonging to a category C only through explicit inheritance declarations (e.g., `class T : public Monoid`). In contrast, `dedekind` adopts a structural type theory. This alignment mirrors the mathematical structuralism advocated by Awodey [7], where mathematical objects are defined not by an internal “essence” but by their roles and relations within a category.

Leveraging C++20/23 Concepts, types are treated as mathematical objects defined entirely by their interface and satisfied invariants. A type T is identified as a `MonoidalCategory` if it possesses the required algebraic morphisms $(\otimes, e)$; its internal name and declaration lineage are irrelevant to the compiler’s logical resolution.

2.1.2 The Compiler as a Verification and Optimization Engine

While functional languages like Haskell are often viewed as the natural home for Category Theory, Milewski [17] demonstrated that C++ is equally capable of modeling complex categorical morphisms. `dedekind` builds upon this bridge by utilizing C++23 features to move from mere simulation to compile-time verification.

By applying `static_assert` against structural concepts, we transform the compilation process into a formal verification phase. This provides the LLVM toolchain with the semantic context required for “structural pruning”: by hoisting mereological relations into the type signature, the backend can identify logical contradictions—such as an empty set defined by contradictory predicates—and prune them into zero-overhead machine instructions. This “correctness-by-construction” methodology ensures that the structural integrity of the mathematical model is verified and optimized before the final binary is generated.

2.1.3 Ontological Decidability and the Build Graph

To mitigate the “template tax” and ensure the practical decidability of the ontology, we utilize a strictly enforced build graph (see Figure 1). This DAG mirrors the formal structure of a mathematical proof: $Axiom \rightarrow Theorem \rightarrow Proof$. While the underlying mathematical nature of certain propositions remains inherently undecidable, `dedekind` provides structural guardrails by isolating foundational axioms into the `:mereology` partition. The resulting BMI acts as the “cached truth” established in the previous section.

This physical stratification prohibits circular reasoning at the compiler level; a partition can only reference previously verified properties. Subsequent partitions, such as `:algebra`, import these BMIs as established lemmas, allowing the compiler to focus on the synthesis of higher-order laws without the risk of recursive definition or redundant re-instantiation. While the module graph serves as an external meta-logical constraint ensuring build-time termination, we later introduce a pluggable *Subobject Classifier* (Ω) in the `:logic` partition (see Section ??). This internalizes the notion of undecidability within the C++ type system itself through a Kleene ternary logic, allowing the library to explicitly represent and propagate indeterminate states without compromising the integrity of the broader derivation.

For the remainder of the implementation section, we follow the strictly enforced build order of the system. We will annotate and document the specific modules and partitions as they are

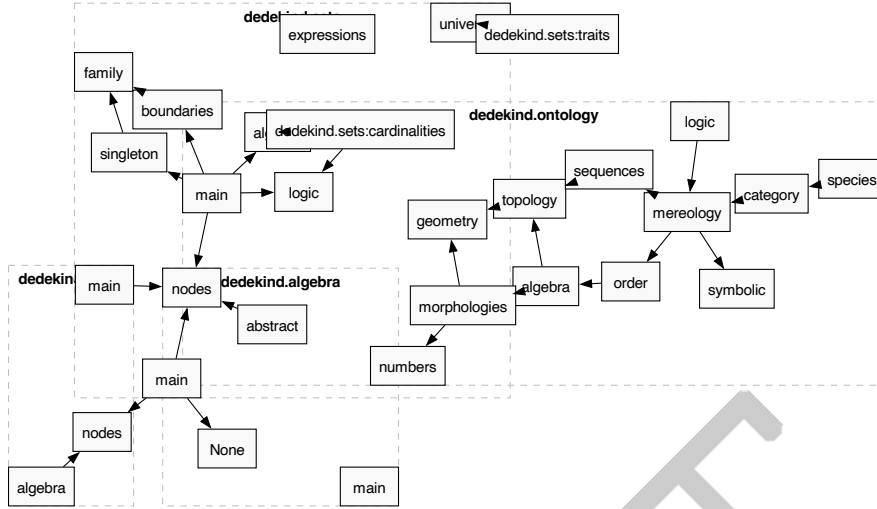


Figure 1: The Ontological Build Graph: Automated dependency resolution.

defined and validated, beginning with the foundational partitions of the `dedekind.category` module.

In this approach, the structure of the paper is intended to be isomorphic to the CI/CD pipeline: the text serves as a formal documentation of the build, while the build, in turn, provides a physical validation of the claims made within the paper. By the time the reader reaches the final sections, the library's 'textbook' derivation will have been fully materialized as a verified C++ library object traversed by the underlying hardware.

2.2 Level 0: The Algebraic Atlas (`dedekind.category`)

The methodology of `dedekind` is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next. The intention is that the flow of compilation mirrors the order in which objects are introduced in a formal textbook: foundational mereological axioms are isolated into the `:mereology` partition, which produces a Binary Module Interface (BMI) that acts as a "cached truth."

2.2.1 The `:species` partition

The `:species` partition functions as an *Algebraic Atlas*, providing a stratified registry of structural invariants for C++ primitive types and standard library operators. By reifying these categorical axioms—associativity, commutativity, and idempotency—as compile-time constants, the library transforms the C++ type system into a formal verification engine. This design follows the combinatorial species theory of Joyal [8, 9], treating fundamental types not merely as sets of values, but as rules for generating structure.

```
/**
 * @tparam T The type (Species) or the Function Object (Arrow).
 * @tparam Args Optional Domain for inference.
 */
export template <typename T, typename... Args>
struct SpeciesTraits;

// ...
```

```

/** @brief Universal inference for any Morphism signature f: Args... -> Codomain
 */
template <typename F, typename... Args>
struct infer_morphism {
    // For binary operators, Domain is the first argument type
    using Domain = std::tuple_element_t<0, std::tuple<Args...>>;
    using Codomain = std::invoke_result_t<F, Args...>;
};

// ...

static_assert(IsCommutative<int, std::plus<int>>,
              "Commutative: Integer addition is commutative.");

// ...

```

Listing 5: Excerpt to illustrate the anchoring the monoid operations as morphisms at compile time.

As shown in Table 1, the "Feature Cube" serves as a formal ledger mapping algebraic properties to the language's fundamental atoms. This explicit mapping allows the `dedekind` dispatcher to distinguish between strict algebraic structures (e.g., `unsigned int` addition) and numerical species, such as floating-point operations which exhibit numerical non-associativity under the IEEE 754 standard [12]. By anchoring these operators as first-class categorical entities, we enable the compiler to act as a physical guardrail for mathematical truth, ensuring structural integrity is verified prior to binary generation.

The overall intention is to anchor the C++ type system (primitive types and operators) as a substrate for modelling truths obtained from category theory and abstract algebra. In this setup, providing the axiomatic truths (e.g. $a||b$ is commutative) or the absence thereof (the messy reality of NaN , overflows, or rounding errors) is through this registry of truths the responsibility of the library developer.

Table 1: The Feature Cube: Structural Invariants for Primitive Species. **Assoc**, **Comm**, and **Idemp** denote algebraic properties of binary operations; **Refl**, **Trans**, and **Anti** denote properties of binary relations; **Period** denotes the "Safety Certificate" for totality (circular topology).

Species	Op / Rel	Assoc	Comm	Idemp	Period	Refl	Trans	Anti	Logic
<i>Logical</i>	<code>std::logical_</code>								
<code>bool</code>	<code>and / or</code>	✓	✓	✓	†	—	—	—	Class.
<code>bool</code>	<code>equal_to</code>	✓	✓	—	—	✓	✓	✓	Class.
<i>Integral</i>	<code>std::bit_</code>								
<code>uint/int</code>	<code>and / or</code>	✓	✓	✓	—	—	—	—	Class.
<code>uint/int</code>	<code>xor</code>	✓	✓	—	✓	—	—	—	Class.
<i>Integral</i>	<code>std::arithmetic</code>								
<code>uint</code>	<code>plus / mult</code>	✓	✓	—	✓	—	—	—	Class.
<code>int</code>	<code>plus / mult</code>	✓	✓	—	—	—	—	—	Class.
<i>Floating</i>	<code>std::arithmetic</code>								
<code>double</code>	<code>plus / mult</code>	*	✓	—	—	—	—	—	Tern.
<i>Relational</i>	<code>std::relational</code>								
<code>uint/int</code>	<code>less_equal</code>	—	—	✓	—	✓	✓	✓	Class.
<code>double</code>	<code>less_equal</code>	—	—	✓	—	‡	✓	✓	Tern.

*Fails bit-perfect associativity due to rounding error in IEEE 754 [12].

†Lattice operations are total via idempotency rather than periodicity.

‡Fails reflexivity ($NaN \leq NaN$ is false).

2.2.2 The :logic partition: The Subobject Classifier (Ω)

Before a "Body" (Set) or a "Path" (Sequence) can be reified, the **dedekind** library establishes the "Rules of Presence" through the **Subobject Classifier** Ω . Following the algebraic logic of Rasiowa [18], we treat every logical system as a specific type of abstract algebra where truth-values act as the objects of a **Topos**. By reifying these systems via the **LogicalSpecies** concept, we prevent the "leaky abstractions" inherent in machine-level **bool** types—such as implicit integral promotion—while allowing for pluggable logical universes.

In the **ClassicalLogic** species, Ω is the binary prime $\{True, False\}$ (see Table 2), corresponding to the standard Boolean Topos where the **Law of Excluded Middle** holds. However, to honestly model computational undecidability or the "truth-holes" inherent in floating-point *NaN* boundaries, we introduce the **TernaryLogic** species. Based on Kleene's strong logic of indeterminacy (K3) [19], this logic introduces an **Unknown** state (0) as seen in Table 2. Within the **dedekind** framework, these logics are formalized as **Rigs**; the *Join* ($\vee$) serves as additive composition (Supremum), while the *Meet* ($\wedge$) serves as multiplicative composition (Infimum). This structural mapping ensures that any **Predicate** $X \rightarrow \Omega$ provides the mathematically rigorous foundation required for the subobject classification in the downstream **:partial** partition.

Table 2: Truth Tables for Classical ($\Omega_{\mathbb{B}}$) and Kleene (Ω_{K3}) Toposes. In the Ternary system, the *Unknown* state (0) acts as a fixed point for negation and a neutral element for disjunction.

Table 3: *

Classical ($\Omega = \{0, 1\}$)

<i>A</i>	<i>B</i>	$A \wedge B$	$A \vee B$	$\neg A$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Table 4: *

Kleene K3 ($\Omega = \{-1, 0, 1\}$)

<i>A</i>	<i>B</i>	$A \wedge B$	$A \vee B$	$\neg A$
-1	-1	-1	-1	1
-1	0	-1	0	1
-1	1	-1	1	1
0	0	0	0	0
0	1	0	1	0
1	1	1	1	-1

The necessity of the **TernaryLogic** species is most evident when observing the breakdown of the **Law of Excluded Middle** ($P \vee \neg P = \top$) in floating-point arithmetic. Following the IEEE 754 standard [12], the NaN (Not-a-Number) state represents a singularity where standard relational morphisms fail to provide a binary partition of the species. Consider a predicate $P(x) := (x \leq 1.0)$ for $x = \text{NaN}$. In classical C++, the expression `(x <= 1.0)` evaluates to **false**. However, the logical complement $\neg P(x)$, represented by `(x > 1.0)`, also evaluates to **false**. This results in the paradox $P \vee \neg P = \perp$, a violation of classical tautology that the **dedekind** library resolves by elevating the subobject classification to Ω_{K3} .

```
double x = std::numeric_limits<double>::quiet_NaN();

// In a Classical Topos (bool), both are false:
bool p      = (x <= 1.0); // false
bool not_p  = (x > 1.0);  // false
bool lem    = p || not_p; // false (LEM Violation!)

// In the Dedekind Ternary Topos (Omega):
Ternary P    = Classifier<double>::eval(x <= 1.0); // Unknown
Ternary NOT_P = !P;                                // Unknown
Ternary LEM   = P || NOT_P;                         // Unknown
```

Listing 6: The NaN Truth-Hole: Violating the Law of Excluded Middle in C++.

By mapping these "numerical hazards" to the **Unknown** state, the library ensures that downstream algebraic transformations (such as those in the `:partial` partition) do not proceed under the false assumption of a binary state. This "honesty" in the logic layer allows the compiler to treat NaN or integer overflow boundaries as formal subobject exclusions rather than undefined behavior.

The **dedekind** library acknowledges the machine's physical constraints by identifying two primary categories of "truth-holes" where the standard mapping between $\mathbb{Z}$ or $\mathbb{R}$ and their machine counterparts, `int` and `double`, breaks down.

The NaN Truth-Hole. As specified in the IEEE 754 standard [12], the NaN state represents a logical singularity. For a predicate $P(x) := (x \leq 1.0)$, if $x = \text{NaN}$, then both $P(x)$ and its negation $\neg P(x)$ evaluate to **false**. This paradox, $P \vee \neg P = \perp$, directly violates the **Law of Excluded Middle (LEM)**, a failure that the **SubobjectClassifier** resolves by elevating the result to the **Ternary Topos** (Ω_{K3}) [19].

The Lipschitz Boundary. For signed integral types, the mapping from the infinite ring of integers to the finite machine representation is only stable within a defined "Lipschitz domain" $[-2^{N-1}, 2^{N-1} - 1]$. Once an operation, such as addition, breaches this boundary, it results in *undefined behavior* or "wrap-around" effects, effectively losing its Lipschitz continuity. By utilizing compiler built-ins (e.g., `__builtin_add_overflow`), the **SubobjectClassifier** detects these boundary breaches and maps them to the **Unknown** state, as illustrated in Listing 7.

```
export template <IsSpecies T>
struct SubobjectClassifier {
    using L = typename LogicTraits<T>::type;
    using Omega = typename L::type;

    /** @brief Lipschitz Boundary Check for Signed Integers */
    template <typename Op>
    requires std::signed_integral<T>
    static constexpr Omega evaluate_arithmetic(T a, T b, Op op) {
        T result;
        if (__builtin_add_overflow(a, b, &result)) {
            return L::Unknown; // Lipschitz boundary breached
        }
        return L::True;
    }

    /** @brief NaN Truth-Hole Check for IEEE 754 */
    template <typename Op>
    requires std::floating_point<T>
    static constexpr Omega evaluate_relational(T a, T b, Op rel) {
        if (std::isnan(a) || std::isnan(b)) {
            return L::Unknown;
        }
        return rel(a, b) ? L::True : L::False;
    }
};
```

Listing 7: Subobject Classifier for the Lipschitz Boundary and NaN Truth-Hole.

2.2.3 The `:morphisms` partition: The Skeletal Arrow

The `:morphisms` partition establishes the primary inhabitant of the **dedekind** topos: the **Arrow** (or Morphism). While traditional C++ development treats functions as opaque executable

blocks, this partition reifies them as formal mappings $f : A \rightarrow B$ equipped with domain and codomain metadata.

Guided Remark: Focus on the definition of the **Morphism** structure and the **IsArrow** concept. This section should explain how we utilize C++23 `std::invoke_result` to perform "Signature Discovery," allowing the library to automatically categorize raw lambdas and function pointers. By establishing this skeletal signature early, we provide the "connective tissue" for the algebraic structures in Level 1 and the functorial lift in Level 2. Document how the `static_assert` checks in this partition verify that a mapping is well-formed before it can be used as a "Taxonomic Brick" in the broader ontology.

2.2.4 The :total partition: The Laws of Harmony

In the "Platonic" domain of our ontology, we define the category $\mathcal{C}$ of **Total Species**, where algebraic structures are governed by a collection of total morphisms $\mu : X \times X \rightarrow X$. Following the *Polish School of Logic* [18], these structures represent "Laws of Harmony" that are treated as absolute invariants. This approach aligns with the *Elementary Theory of the Category of Sets* (ETCS) as proposed by Lawvere [20, 21], where types are defined by their morphisms rather than their internal elements [10].

As shown in Table 5, species in this partition "mature" as they gain axioms, moving from the primal **Magma** to the perfect symmetry of an **Abelian Group**. A primary inhabitant of this total universe is the `bool` species; under the morphisms of logical conjunction ($\wedge$) and disjunction ($\vee$), it behaves as a **Distributive Lattice**—a specialized **Rig** where both operations are **Idempotent**. This idempotency provides the structural basis for **Universal Constructions** such as *Products* and *Coproducts*, which appear in systems programming as *Meet* and *Join* operations [17].

By mapping these structures to C++23 concepts, the `dedekind` library transforms the compiler into a **Constraint Satisfaction Engine** (CSE) that verifies mathematical truth as a physical invariant of the binary [22]. This leads to the **Honest Rejection** illustrated in Listing 8: while `unsigned int` addition is verified as a total Abelian Group ($\mathbb{Z}/2^n\mathbb{Z}, +$), signed `int` addition is rejected as a total Magma due to the potential for overflow at the Lipschitz boundary.

Table 5: The Path to Symmetry: Single-Operation Species in `dedekind.category`. Each level introduces a new structural invariant, from basic closure to commutative inversion.

Concept	Structure	Axiomatic Requirements	C++ Concept
Magma	$\langle T, \circ \rangle$	$\forall a, b \in T : a \circ b \in T$ (Totality)	<code>IsMagma</code>
Semigroup	$\langle T, \circ \rangle$	Magma + Associativity	<code>IsSemigroup</code>
Monoid	$\langle T, \circ, e \rangle$	Semigroup + Identity	<code>IsMonoid</code>
Group	$\langle T, \circ, e, \text{inv} \rangle$	Monoid + Inverse	<code>IsGroup</code>
Abelian Group	$\langle T, \circ, e \rangle$	Group + Commutativity	<code>IsAbelianGroup</code>

```
// Level 1.3: The Perfect Symmetry (Monoid + Inverse + Symmetry)
export template <typename T, typename Op>
concept IsAbelianGroup = IsMonoid<T, Op> &&
                        IsLoop<T, Op> &&
                        IsCommutative<T, Op>;

/** @section Truth_Anchors */

// SUCCESS: Unsigned addition is a Total Abelian Group (Z/2^nZ).
static_assert(IsAbelianGroup<unsigned int, std::plus<unsigned int>>);

// HONEST REJECTION: Signed addition is NOT a Total Magma.
// It fails because of Undefined Behavior/Overflow (Partiality).
```

```
| static_assert(!IsMagma<int, std::plus<int>>>);
```

Listing 8: Structural Maturation and the Honest Rejection in `dedekind.category`.

2.2.5 The `:functor` partition: Mappings between Categories

Following the fractal progression of the library, the `:functor` partition reifies the algebraic laws of Level 1 to act upon the categories themselves. A Functor is treated not as a container, but as a structural morphism $F : \mathcal{C} \rightarrow \mathcal{D}$ that preserves the composition of arrows and identity mappings.

Guided Remark: Document the implementation of `fmap` as a high-level dispatcher. This section should highlight how the `dedekind` dispatcher utilizes C++23 template-template parameters to ensure that functorial mapping is a zero-overhead operation, effectively "shielding" the user from the underlying machine representation while maintaining categorical rigor.

2.2.6 The `:natural` partition: Natural Transformations

To complete the Level 2 bridge, the `:natural` partition introduces Natural Transformations—morphisms between functors. This provides the mathematical basis for "Component-wise" transformations that remain invariant under functorial mapping, satisfying the naturality square: $\alpha_Y \circ F(f) = G(f) \circ \alpha_X$.

Guided Remark: This partition is a technical prerequisite for the `:monad` partition, as it provides the formal definition for the unit (η) and multiplication (μ) transformations. Document how `dedekind` utilizes C++23 Concepts to verify the "naturality" of a transformation at compile-time, ensuring that structural mappings are consistent across different functorial contexts.

2.2.7 The `:monad` partition: The Ω -Monad

While Level 1 defines absolute laws, real-world computation requires a mechanism for handling "effectful" transformations. Following the constructions of Kleisli [23] and Moggi [6], we implement the **Kleisli Triple** ($\eta, \gg=$) as the fundamental unit of composition. In the `:monad` partition, we unify the "Rules of Presence" from the `:logic` topos with this machinery to define the Ω -Monad.

As illustrated in Listing 9, we extract the `IsPotential` concept to create a structural isomorphism between standard C++ types and the `dedekind Partial<T>` container. To resolve the lack of native higher-kinded types (HKTs) in C++, we utilize an "Action-First" bootstrapping strategy: the `fmap` functorial mapping is derived as an *epi-phenomenon* of the underlying monadic "Push" ($m \gg= \eta \circ f$). This ensures that any species providing a `bind` operator ($\gg=$) automatically matures into a *Functor* through what we term *Synthetic Inference*.

```
export template <typename R>
concept IsPotential = requires(R r) {
    typename R::value_type;
    { *r } -> std::convertible_to<typename R::value_type&>;
    { presence_of(r) } -> LogicalValue; // Bridge to Omega
};

// The Universal Highway: Works for any IsPotential result
export template <IsPotential M, typename F>
constexpr auto operator>>=(const M& m, F f) {
    using L = typename GetLogic<M>::type;
    if (presence_of(m) != L::True) return empty<M>(presence_of(m));
    return f(*m);
}
```

Listing 9: The Ω -Monad: Structural Parity and the Universal Highway.

2.2.8 The :kleisli partition: The Universal Highway

A core innovation of the Kleisli bridge is the introduction of the “**Highway**” notation (`operator»`) for Kleisli composition. Grounded in the functional tradition of the “Fish Operator” (`>=>`) [24], `dedekind` adapts this to the C++ context by leveraging the left-associativity and higher precedence of `operator»` over the standard monadic bind (`operator>=`).

As advocated by Milewski [17], viewing monads as a composition engine for “embellished functions” allows the compiler to treat the entire Highway as a single, verifiable unit of transformation. Within the verified module graph, the LLVM backend can perform *structural pruning*: if a total function is composed with a partial operation statically known to return an **Unknown** or **NaN** state, the compiler can identify the unreachable path and optimize the Highway into a branch-free instruction sequence.

```
// Equivalent to Haskell's (f >=> g)
auto highway = morphism_f >> morphism_g;

// Composition is verified against the Monad axioms
static_assert(IsKleisliArrow<decltype(highway)>);
```

Listing 10: The Highway: Kleisli composition as a first-class citizen.

2.2.9 The :partial partition: The Pragmatic Rescue

The `:partial` partition performs the “Categorical Rescue” of species that failed the strict requirements of the `:total` domain. This stage formalises the transition from the *Platonic* to the *Pragmatic*: the goal is to show that while the machine is “broken,” it is **consistently broken**. Following the informal explorations of the *n-Category Café* [11] and the subsequent formalisations of the *nLab* community [25], we acknowledge these consistent failures—such as the Lipschitz boundary of signed integers or the truth-holes of IEEE 754—not as errors, but as structured data.

Following topos-theoretic foundations [19], we treat a partial morphism $f : A \multimap B$ as a total mapping from a *subobject* $S \hookrightarrow A$. Here, S represents the **Support** defined by the characteristic function $\chi : A \rightarrow \Omega_{K3}$ provided by the `:logic` partition. Within this restricted domain, we can define *locally total* structures, such as **Partial Groups** or **Partial Loops**. These structures guarantee that the “Laws of Harmony” hold strictly on the support $U = \{a \in A \mid \chi(a) = \top\}$, ensuring that even when navigating the messy reality of hardware, we remain within a rigorous algebraic landscape.

By reifying the `Partial<T>` “Rescue Pod” as an inhabitant of the **Ω -Monad**, the library enables the composition of these partial morphisms via the universal highway established in the `:monad` partition. The bind operator (`>=>`) acts as the monadic gatekeeper: it consumes the **Ternary** presence, ensuring that any “Truth-Hole” is propagated through the Kleisli chain. This transforms “Consistently Broken” machine behavior into a rigorous, composable algebraic flow, preventing undefined machine behavior while maintaining the structuralist integrity of the transformation pipeline.

```
// The "Rescue": int addition is recognized as a Partial Abelian Group.
static_assert(IsPartialAbelianGroup<int, std::plus<int>>);

// Composition in the Kleisli Category:
auto f = partial_plus(1, 2);           // Result: 3 (True)
auto g = partial_plus(f, INT_MAX);     // Result: Unknown (Overflow)
auto h = partial_plus(g, 1);           // Result: Unknown (Propagation)
```

Listing 11: The Kleisli Lift: Reclaiming the Lipschitz Boundary via Subobject Exclusion.

2.2.10 The `:numeric` partition: The Material Landing

The `:numeric` partition represents the final “landing” of the abstract theory back onto the hardware substrate. While the `:partial` partition establishes the general algebraic laws for inconsistent structures, the `:numeric` partition provides the specific mapping for IEEE 754 floating-point types and bounded integers.

Guided Remark: Focus on how the library utilizes the `Partial<T>` rescue pod to formally model the non-associative nature of `double` and the non-reflexive behavior of `NaN`. This section should document the “Safety Certificates” for specific machine species, explaining how the `dedekind` dispatcher uses these certificates to enable or disable algebraic optimizations. By anchoring these hardware-specific behaviors as first-class inhabitants of a partial monoid, we ensure that the “messy reality” of rounding errors and overflows is not an exception to the theory, but a verified and traceable component of the categorical flow.

Finally, we encounter the “Curious” case of *Numerical Species* (IEEE 754). These types are categories where the subobject classifier Ω is internalized within the species itself (`NaN`, `Inf`). While these structures appear to be total morphisms $\mu : \mathbb{R}_f \times \mathbb{R}_f \rightarrow \mathbb{R}_f$ in their signature, they are semantically partial. Specifically, they fail the **Equalizer** conditions required for associativity; the rounding error ensures that $(a + b) + c$ and $a + (b + c)$ are not generally isomorphic. This progression—from the absolute (*Total*), through the filtered (*Partial*), to the exceptional (*Numerical*)—allows us to build a “strictly speaking” true ontology that respects both categorical rigor and machine-level reality.

2.2.11 The Semantic Mapping

The `dedekind.category` module serves as an anchor of the DSL in the established formalism of category theory. It aggregates skeletal, mereological, and algebraic laws to enable the synthesis of the *Continuum* from the *Discrete*. The adopted *structuralist* posture views mathematical objects as positions within a structure rather than isolated collections. McLarty [19] demonstrated that this is sufficient for the complete recovery of number theory. In turn, we facilitate formal verification within the C++ type system with zero runtime overhead.

Note the pain with C++ “transparent” operator mappings in the truth registry.

The stratification of the `dedekind.ontology` into C++ module partitions mirrors the constructive dependencies of formal mathematics. At the foundation, an embedding of Category Theory (Level 0) within the type system provides the primary substrate; as it makes the fewest assumptions—treating objects merely as nodes in a graph of morphisms—it establishes the “highways” (Functors and Kleisli Triples) necessary for all subsequent synthesis. Because higher algebraic structures—such as Groups, Rings, and Fields—are formally defined by the operations performed upon sets, the build order mandates that Level 1 (Space and Mereology) precedes Level 3 (Algebra). This ensures that the “Soul” of the system (its laws of action and harmony) is always grounded in the “Body” (the structural presence of its parts). By the time the compiler reaches the `:algebra` partition, the mereological validity of the underlying species is already a type-checked invariant within the translation unit. This hierarchy transforms the C++ module mapper into a directed acyclic graph of mathematical truth, where the linker cannot even begin its work until the ontological prerequisites are satisfied. The strict directed acyclic graph (DAG) of the `dedekind` module partitions mirrors the Bourbaki style of ‘Mother Structures’ [26], where the physical order of compilation serves as a proof of logical consistency.

2.3 sets Module

For the set operations, we may also want to look at: “Monadic Intersection Types, Relationally, and Ordered” <https://dl.acm.org/doi/10.1145/3777483> ... and references cited therein.

2.3.1 The :set partition

To ground this theoretical approach, Table 1 maps the fundamental axioms of the Elementary Theory of the Category of Sets (ETCS) – as codified in the pedagogical treatment by Lawvere and Rosebrugh [27] – to their structural equivalents in `dedekind`. This mapping ensures that the library is not merely a numerical simulation, but a direct implementation of algebraic laws within the host language’s type system. In a sense, `dedekind` hopes to be a **compiler** for mathematical concepts.

Table 6: Complete Mapping of ETCS Axioms to C++23 Structural Invariants

ETCS Axiom	C++ Language Feature	Structural Invariant
1. Composition	<code>std::invoke / operator></code>	<code>IsSemigroupoid<T, Op></code>
2. Identity	<code>identity_trait<T, Op></code>	<code>IsSmallCategory<T, Op></code>
3. Terminal Object	<code>s(x) -> Logic::top</code>	<code>IsTerminalObject<T></code>
4. Well-Pointedness	<code>operator()</code> (Element access)	<code>IsWellPointed</code>
5. Cartesian Product	<code>std::tuple / std::pair</code>	<code>HasProduct<A, B></code>
6. Exponentiation	Lambda NTTP / <code>std::function</code>	<code>IsInternalHom<A, B></code>
7. Equalizers	<code>dedekind::set<T, P></code>	<code>IsEqualizer<f, g></code>
8. Empty Set	<code>dedekind::EmptySet</code>	<code>IsInitialObject<T></code>
9. Infinity (NNO)	<code>dedekind::Naturals</code>	<code>HasNaturalNumbersObject</code>
10. Axiom of Choice	Lattice Dispatcher	<code>AxiomOfChoice</code>

By adopting a categorical foundation, the library provides a unified interface for both *extensional* and *intensional* representations. For finite collections, such as the `Booleans`, the system performs *extensional* materialization to compute exact results at compile-time. Conversely, for infinite or transcendental structures like the `Complex Numbers`, the library utilizes *intensional* symbolic expressions to maintain structural integrity. By embedding these categorical definitions in the C++ host language, the programmer—and crucially the *compiler*—can examine the algebraic properties of a field even when the underlying elements cannot be fully materialized as discrete machine data.

2.3.2 Intensional Sets and Computational Undecidability

The utility of the categorical approach is most evident when defining sets whose membership conditions are conceptually clear but computationally undecidable. Consider the set of *Normal Numbers* $\mathcal{N}$, defined as those real numbers whose infinite digit expansion follows a uniform distribution. Formally, a real number x is normal in base b if for every finite sequence of digits s , the limiting frequency of s appearing in the expansion of x satisfies:

$$\lim_{n \rightarrow \infty} \frac{N(s, x, n)}{n} = \frac{1}{b^{|s|}} \quad (1)$$

where $N(s, x, n)$ denotes the number of occurrences of the string s in the first n digits of x . While we can easily define this property as a symbolic predicate within the `dedekind` ontology, the membership of specific constants like π or e remains a famous open problem in mathematics.

In a traditional imperative framework, such a set is unusable because membership cannot be *computed* to a boolean result. However, by treating $\mathcal{N}$ as an *intensional* set, the `dedekind` library allows the programmer to define morphisms that take $\mathcal{N}$ as their formal domain. This enables the C++ compiler to verify the structural validity of a function pipeline based solely on the *definition* of normality, even while the empirical membership of π remains unresolved. By hoisting the predicate into a **Stateless Constraint Type**, we decouple the logical requirement of the set from the necessity of materializing its elements. This allows the compiler to treat $\mathcal{N}$ as a valid algebraic object, facilitating symbolic reasoning about digit distribution without

requiring a prior numerical proof of normality, thereby avoiding the runtime overhead of high-precision digit evaluation.

This allows the compiler to treat $\mathcal{N}$ as a valid algebraic object, facilitating symbolic reasoning about digit distribution without requiring a prior numerical proof of normality, thereby avoiding the runtime overhead of high-precision digit evaluation.

This intensional logic extends to non-elementary functions where the antiderivative cannot be expressed in a finite combination of elementary terms. For example, the Gaussian integral e^{-t^2} defines the *Error Function* (erf) as a structural rule for area rather than a simple algebraic formula:

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \quad (2)$$

While traditional backends rely on lossy polynomial approximations to evaluate this integral, the **dedekind** ontology reifies $\text{erf}(x)$ as a set of symbolic properties. This allows the **C++** compiler to resolve identities—such as the symmetry $\text{erf}(-x) = -\text{erf}(x)$ —as zero-overhead type transformations, delaying numerical approximation until the final realization of the continuum.

2.3.3 Categorification of Sequences and the Continuum

While ETCS provides the axioms for discrete collections, the representation of sequences and the analytic continuum requires a transition from set-theoretic membership to functorial structures. In the **dedekind** library, we avoid the imperative indexing of terms by treating sequences as *Combinatorial Species* [8]. By defining a sequence as a functor from the category of finite sets to itself, we can express growth rules—such as the Fibonacci recurrence—as algebraic identities between functors. This allows the compiler to resolve the structural identity of a series through symbolic substitution before any machine-level summation occurs [9]. To reach the exact real arithmetic promised in Section 1, we ground our implementation of convergence in LAWVERE’s theory of enriched categories [28]. By treating metric spaces as categories enriched over the poset of non-negative reals $[0, \infty)$, the library reifies Cauchy sequences not as floating-point approximations, but as formal Cauchy completions [29]. This methodology ensures that the *Dedekind cut*—defining real numbers $\mathbb{R}$ as partitions of the rationals $\mathbb{Q}$ [27]—is not merely a runtime filter, but a verified limit of a rational sequence. This allows the LLVM backend to treat transcendental constants like e and π as symbolic invariants rather than opaque numeric variables.

The library addresses the representation of transcendental constants by distinguishing between their algebraic *definition* and their numerical *realization*. While the machine may be unable to *compute* the final digit of π , the **dedekind** ontology can lazily *define* it as a formal limit of a rational sequence. By encoding the LEIBNIZ series for π or the TAYLOR expansion for e , the library treats these constants as symbolic *intensional* types.

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + 1 + \frac{1}{2} + \frac{1}{6} + \dots \quad (3)$$

This approach leverages *lazy evaluation* at the type-level, where the **C++** compiler performs symbolic substitutions of the series terms before any floating-point approximation is required. By teaching the compiler the *rule* of the sequence rather than a static value, the library ensures that mathematical identities—such as $e^0 = 1$ —can be resolved as zero-instruction compile-time proofs.

xxxxxx

While ETCS provides the axioms for discrete collections, the representation of sequences and the analytic continuum requires a transition from set-theoretic membership to functorial structures. In the **dedekind** library, we avoid the imperative indexing of terms by treating sequences as *Combinatorial Species* [8]. By defining a sequence as a functor from the category of

Table 7: Categorification of Sequences and Series in the Dedekind Ontology

Mathematical Concept	Set-Theoretic Form	Categorical Framework	C++ Structural
Recursive Sequence	$F_n = F_{n-1} + F_{n-2}$	Combinatorial Species [8]	Functorial Coprodu
Formal Power Series	$\sum a_n x^n$	Analytic Functors [9]	Polynomial Species
Metric/Distance	$ x - y < \epsilon$	Lawvere Enriched Category [28]	Enriched Hom-sets
Cauchy Convergence	$\lim_{n \rightarrow \infty} a_n$	Cauchy Completion [29]	Limit / Terminal O
Real Numbers ($\mathbb{R}$)	Dedekind Cuts / Cauchy	Archimedian Completion [19]	Adjoint Functors

finite sets to itself, we can express growth rules—such as the Fibonacci recurrence—as algebraic identities between functors. This allows the compiler to resolve the "combinatorial essence" of a series through symbolic substitution before any machine-level summation occurs [9]. To reach the exact real arithmetic promised in Section 1, we ground our implementation of convergence in Lawvere’s theory of enriched categories [28]. By treating metric spaces as categories enriched over the poset of non-negative reals $[0, \infty)$, the library reifies Cauchy sequences not as floating-point approximations, but as formal Cauchy completions [29]. This methodology ensures that the Dedekind cut – defining real numbers as partitions of the rationals [27] – is not merely a runtime filter, but a verified limit of a rational sequence, allowing the LLVM backend to treat transcendental constants like e and π as first-class structural invariants.

The library addresses the representation of transcendental constants by distinguishing between their algebraic *definition* and their numerical *realization*. While the machine may be unable to *compute* the final digit of π , the `dedekind` ontology can lazily *define* it as a formal limit of a rational sequence. By encoding the Leibniz series for π or the Taylor expansion for e , the library treats these constants as symbolic *intensional* types.

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + 1 + \frac{1}{2} + \frac{1}{6} + \dots \quad (4)$$

This approach leverages **lazy evaluation** at the type-level, where the C++ compiler performs symbolic substitutions of the series terms before any floating-point approximation is required. By teaching the compiler the *rule* of the sequence rather than a static value, the library ensures that mathematical identities—such as $e^0 = 1$ —can be resolved as zero-instruction compile-time proofs.

Table 8: Categorification of Sequences and Series in the Dedekind Ontology

Mathematical Concept	Set-Theoretic Form	Categorical Framework	C++ Structural
Recursive Sequence	$F_n = F_{n-1} + F_{n-2}$	Combinatorial Species [8]	Functorial Coprodu
Formal Power Series	$\sum a_n x^n$	Analytic Functors [9]	Polynomial Species
Metric/Distance	$ x - y < \epsilon$	Lawvere Enriched Category [28]	Enriched Hom-sets
Cauchy Convergence	$\lim_{n \rightarrow \infty} a_n$	Cauchy Completion [29]	Limit / Terminal O
Real Numbers ($\mathbb{R}$)	Dedekind Cuts / Cauchy	Archimedian Completion [19]	Adjoint Functors

This mapping ensures that our software implementation is not merely a *simulation* of set theory, but a direct *instantiation* of its algebraic laws.

The structural mereology [30] approach in `dedekind` defines sets via formal relations—union $(X \cup Y)$, intersection $(X \cap Y)$, and parthood $(X \subseteq Y)$ and last but not least element $(x \in Y)$ – prior to implementation. This mirrors structuralist foundations such as Lawvere’s Elementary Theory of the Category of Sets (ETCS) [21] and Cartesian Closed Categories (CCCs) [31, 32]. By prioritizing *structural transparency*, this methodology fundamentally departs from object-oriented encapsulation. In a sense, it encodes the well known rule of thumb of "composition

over inheritance" [26]. In a standard `std::set`, the internal representation is an opaque implementation detail, invisible to the type system's logic. Conversely, in `dedekind`, the *type is the definition*. Because the set's constituents—predicates, universes, and mereological relations—are hoisted into the type signature as first-class members, the C++ type system and the CMake toolchain function as a Constraint Satisfaction Engine (CSE). This allows the compiler to inspect, decompose, and optimize a set based on its logical essence rather than its storage layout, trading the 'black-box' safety of encapsulation for the 'white-box' provability of formal mereology.

```
#include <concepts>
#include <type_traits>

// Verifies the "Element Of" relation (x in Y)
template <typename T, typename S>
concept IsElementOf = requires(T x, S s) {
    { s.contains(x) } -> std::same_as<bool>;
};

// Verifies "Parthood" or "Subset" relation (X subset of Y)
template <typename Sub, typename Super>
concept IsSubsetOf = requires {
    requires std::derived_from<Sub, Super> ||
    requires { typename Sub::is_subset_of<Super>; };
};

// CSE Pruning: Collapsing contradictory intersections to EmptySet
template <typename SetA, typename SetB>
struct Intersection {
    using type = std::conditional_t<
        HasContradictoryPredicates_v<SetA, SetB>, // Evaluated at compile-time
        EmptySet,
        InternalIntersection<SetA, SetB>
    >;
};
```

Listing 12: Compile-time mereological verification using C++23 concepts.

By reifying a subset of the set-builder notation, `dedekind` turns the C++ translation unit into a symbolic laboratory. The programmer is free to reason in the language of Cantor, while the compiler produces the performance of a naked C++ loop.

2.4 order Module

2.5 topology Module

2.6 sequences Module

2.7 algebra Module

Needed to push back repeatedly on attempts to go the convenient 'double' route instead of spelling things out as semi-modules, semi-rings etc. i.e. reduced-strength algebraic structures. Of course, ultimately everything is a complex number from this perspective just like everything is a string for front-end developers.

2.8 morphologies Module

2.9 geometry Module

2.10 numbers Module

2.11 analysis Module

The physical build order of the `dedekind.ontology` module mirrors this conceptual hierarchy. By enforcing a linear dependency graph—where `:mereology` is compiled before `:algebra`—we ensure that the compiler’s proof-search for a Ring’s identity element is grounded in the prior verification of its mereological subobjects. This hierarchy transforms the C++ module mapper into a directed acyclic graph of mathematical truth.

2.12 The Compiler as a Deterministic Proof Assistant

The core of `dedekind` is a recursive template evaluator that treats C++ `concepts` as logical predicates. Unlike traditional template metaprogramming that relies on SFINAE, we utilize C++23’s `requires` expressions to perform "structural inspection" of types.

In the `dedekind` framework, the C++ compiler acts as a decidability engine for our mathematical ontology. Rather than relying on traditional class hierarchies, we use C++23 `Concepts` to define the structural requirements of a species. A type is not explicitly inherited from a base; instead, it is verified to be compliant with a species’ `concept` through its intrinsic properties.

This verification is supported by Template Specialization to define axiomatic base cases and SFINAE to prune invalid overloads. We leverage the Curiously Recurring Template Pattern (CRTP) within our internal wrappers to provide a unified interface for disparate types without the overhead of dynamic dispatch. By conducting these checks in a `constexpr` context and utilizing Move Semantics for resource management, we ensure that the "Proof" of a mathematical construction is resolved entirely at compile-time.

```
// Verifying that the intersection of disjoint sets is empty
using Evens = dedekind::set<int, [](auto x){ return x % 2 == 0; }>;
using Odds  = dedekind::set<int, [](auto x){ return x % 2 != 0; }>;

using Intersection = dedekind::meet_t<Evens, Odds>;

// The compiler resolves this to the Empty Set species
static_assert(dedekind::is_empty_v<Intersection>);
static_assert(sizeof(Intersection) == 1); // No machine state required
```

Listing 13: LCompile-time inference of set invariants

2.13 The Semantic Mapping

Crucially, the `dedekind.ontology` acts as the definitive mapping between formal mathematical nomenclature and the concrete syntax of the C++23 language. Rather than introducing a foreign vocabulary, we anchor established axioms directly into the standard operator set and primitive types. For instance, the mereological *Sum* and *Product* are reified as the bitwise operators `|` and `&`, while the *Mereological Complement* (the remainder) is bound to the negation operator `!`.

By establishing this explicit correspondence—where `operator<=` serves as the formal proof of *Parthood* ($\sqsubseteq$) and `operator/` represents the *Quotienting* of a species—we ensure that the resulting code is not merely a simulation of a mathematical model, but a direct execution of it. This semantic alignment allows the programmer to write expressions that are readable as equations, while the compiler treats them as a sequence of verifiable type-transformations. In this architecture, the C++ `concept` becomes the "Gatekeeper of Truth," ensuring that a species

cannot participate in an operation (such as a Join-Semilattice union) unless it has first provided a manual or structural certification of its algebraic invariants.

2.14 The Registry of Structural Proofs

The transition from a raw computational graph to a simplified mathematical identity requires a formal "Seal of Truth." In `dedekind`, we implement a *Traits Registry* that forces the developer to explicitly certify the equational axioms of a species before it can participate in the lattice-based optimizations of the dispatcher.

This is achieved through a dual-layered system of **Trait Ledgers** and **Discovery Concepts**. For each fundamental algebraic law, we define a primary template (the ledger) which defaults to `false`:

```
export template <typename T, typename Op>
struct is_associative : std::false_type {};

export template <typename T, typename Op>
inline constexpr bool is_associative_v = is_associative<T, Op>::value;
```

Listing 14: The Associativity Ledger in `:species`

A species S is only recognized as a `IsSemigroupoid` if it (or the ontology) provides a specialization of this ledger. To allow for "Interoperable Discovery," we provide a fallback mechanism that probes the species for internal certification:

```
export template <typename T, typename Op>
concept IsAssociative = IsMagmoid<T, Op> && requires {
    requires is_associative_v<T, Op>;
};

// Discovery Fallback: Probing the Species for an Intrinsic Proof
template <typename T, typename Op>
    requires requires { T::is_associative_for<Op>; }
struct is_associative<T, Op> : T::template is_associative_for<Op> {};
```

Listing 15: The Concept of Formal Association

By anchoring concepts like `IsJoinSemilattice` in these requirements (requiring both `IsAssociative` and `IsIdempotent`), we ensure that the *Lattice Dispatcher* never performs a "speculative" pruning. If the developer has not provided the proof, the dispatcher falls back to a safe, unoptimized `UnionSet` synthesis. This "Axiom of Responsibility" ensures that the performance of the "Naked Loop" is always mathematically justified by a prior formal certification.

2.15 Ontological Model Validation

To verify the integrity of the proposed mapping between formal axioms and C++ concepts, we employ a "Reverse Curry-Howard" validation strategy. Rather than simply assuming the correctness of our ontological translations, we use the inherent properties of C++ built-in types as an empirical proof of the reverse-engineered axioms.

By executing a suite of `static_assert` evaluations on primitive types—such as verifying that `bool` satisfies the requirements of an `IsJoinSemilattice` or that bitwise operations on `uint32_t` adhere to the `IsMereologicalLattice` consistency axioms—we confirm that the machine-level behavior is isomorphic to the formal mathematical theory. In this posture, the built-in types serve as the "Initial Models" for our concepts. If the compiler successfully verifies that a standard integer under bitwise conjunction satisfies our `IsMeetSemilattice` proof, it provides an automated, formal confirmation that our C++ concepts correctly capture the essential structural invariants found in the mathematical literature.

Table 9: The Dedekind Semantic Mapping: A Registry of Formal Correspondences

Mathematical Concept	Symbol	C++ Operator / Type	C++ Concept	Basis
<i>Category Theory</i>				
Kleisli Extension	$(T, \eta, \gg=)$	<code>operator>=</code>	<code>IsKleisliExtension</code>	<code>IsAssociative</code>
Kleisli Extension	$\gg=$	<code>operator>=</code>	<code>IsKleisliExtension</code>	<code>IsAssociative</code>
Kleisli Composition	$\circ_K$	<code>operator></code>	<code>IsMonad</code>	<code>IsAssociative</code>
Co-Kleisli Extension	$\gg_{\circ}$	<code>operator<=</code>	<code>IsComonad</code>	<code>IsAssociative</code>
Morphic Injection (Unit)	η	<code>into<F> / singleton</code>	<code>IsMonad</code>	$\eta : T \rightarrow F(T)$
Morphic Extraction (Counit)	ε	<code>extract<F></code>	<code>IsComonad</code>	$\varepsilon : F(T) \rightarrow T$
Characteristic Morphism	χ	<code>operator()</code>	<code>IsCharacteristic</code>	
Co-Kleisli Composition	$\circ_{CoK}$	<code>operator<</code>	<code>IsComonad</code>	
Counit (Extraction)	ε	<code>extract<F></code>	<code>IsComonad</code>	
<i>Mereology</i>				
Proper Parthood	χ	<code>operator()</code>	<code>IsProperPart</code>	<code>IsCharacteristic</code>
Parthood (Pre-order)	$\sqsubseteq$	<code>operator<=</code>	<code>IsPartOf</code>	
Mereological Sum (Join)	$\sqcup$	<code>operator </code>	<code>IsJoinSemilattice</code>	
Mereological Product (Meet)	$\sqcap$	<code>operator&</code>	<code>IsMeetSemilattice</code>	
Universal Complement	$\neg$	<code>operator!</code>	<code>IsComplemented</code>	
Initial Object (Empty)	$\emptyset$	<code>$\emptyset<T>$</code>	<code>IsInitialObject</code>	
Terminal Object (Universal)	Ω	<code>$\Omega<T>$</code>	<code>IsTerminalObject</code>	
<i>Set Theory</i>				
Membership	$\in$	<code>operator()</code>	<code>IsSet</code>	<code>IsProperPart</code>
Subset	$\subseteq$	<code>operator<=</code>	<code>IsSubset</code>	<code>IsPartOf</code>
Countable Infinity	$\aleph_0$	<code>$\aleph_0$</code>	<code>IsCardinality</code>	
Continuum Cardinality	$\beth_1$	<code>$\beth_1$</code>	<code>IsCardinality</code>	
<i>Order Theory</i>				
Pre-order	$\sqsubseteq$	<code>operator<=</code>	<code>IsPartiallyOrdered</code>	<code>IsPartOf</code>
<i>Algebra</i>				
Structural Origin (Zero)	0	<code>T::origin()</code>	<code>IsPointed</code>	
Additive Composition	+	<code>operator+</code>	<code>IsGroup</code>	
Additive Inverse	-	<code>operator-</code>	<code>IsGroup</code>	
Multiplicative Composition	$\times$	<code>operator*</code>	<code>IsRing</code>	
Multiplicative Inverse	$/, ^{-1}$	<code>operator/</code>	<code>IsField</code>	
Associativity	$(a \circ b) \circ c$	<code>is_associative_v</code>	<code>IsSemigroupoid</code>	
Idempotency	$a \circ a = a$	<code>is_idempotent_v</code>	<code>IsIdempotent</code>	
Magnitude / Count	$ S $	<code>s.size() / s.cardinality()</code>	<code>IsCardinality</code>	
Supremum / Infimum	$\sup, \inf$	<code>s.supremum(), s.infimum()</code>	<code>HasExtrema</code>	

3 Implementation: The Functor Highway

The operational efficiency of **dedekind** is achieved through a technique we term *Synthetic Inference*. This level represents the "Skeletal Foundation" of the ontology, where machine-level instructions are unified with the abstract laws of Category Theory. By treating the C++ type system as a formal proof assistant, we ensure structural integrity via compile-time verification, or *Static Mereology*.

3.1 The Action-First Bootstrapping Strategy

In textbook Category Theory, a Monad is traditionally defined as a *Monoid in the category of Endofunctors*. However, the structural implementation of this definition in C++ faces significant language constraints:

1. **Partial Specialization Limitations:** C++ forbids the partial specialization of function templates (e.g., `fmap<F>`), preventing a centralized, generic definition for disparate species.
2. **ADL Resolution:** Function templates called via explicit name-lookup cannot trigger Argument-Dependent Lookup (ADL), making it difficult for the ontology to "discover" mappings for types defined in downstream modules.

To resolve this circular dependency, **dedekind** implements an *Action-First* bootstrapping strategy. By defining Monads and Comonads as *Extension Systems* (Kleisli Triples consisting of η/ϵ and $\gg = / \ll =$), we utilize operator overloading on concrete objects. This triggers ADL at the call site, allowing the Level 0 foundation to instantiate a derived `fmap` without prior knowledge of Level 1 species.

3.2 Highway Notation: Directional Vectors

To facilitate a readable "Algebra of Programming," we map categorical data flows to directional operators, creating a "Highway Notation" that reflects the vector of computation across the ontology:

- `value » into<F>` : Represents η (Unit), lifting a species into a context.
- `box « extract<F>` : Represents ϵ (Counit), sampling a species from a context.
- `box »= f` : Represents *Bind*, the forward monadic composition (Left-to-Right).
- `box «= f` : Represents *Extend*, the contextual comonadic composition (Right-to-Left).

```
// Convert plain lambdas into tagged endomorphisms:
auto f = endo<int>([](int x) { return x + 1; });
auto g = endo<int>([](int x) { return x * 2; });

// Morphism composition: (h = g . f):
auto h = f >> g;

// Lifting into the Box and applying the composed morphism:
auto result = 5 >> into<Box> >> fmap<Box>(h);

CHECK(result == Box<int>{12});
```

Listing 16: Example of fused Kleisli "Highway" composition via the functor law.

```

// Define two monadic arrows (Species -> Boxed Species)
auto f = [](int x) { return pure(x + 1); };
auto g = [](int x) { return pure(x * 2); };

// Chain composition via Bind (>=)
// Starting with a raw value, lifting it, then chaining the monadic arrows.
auto x = (5 >> into<Box> >>= f) >>= g;

// (5 + 1) * 2 = 12, wrapped in a Box
CHECK(x == Box<int>{12});

// Verification of the Monadic Associativity Law:
// (m >>= f) >>= g is equivalent to m >>= (x => f(x) >>= g)
auto lh = (5 >> into<Box> >>= f) >>= g;

auto rh = 5 >> into<Box> >>= [&](int x) { return f(x) >>= g; };

CHECK(lh == rh);
STATIC_CHECK(std::same_as<decltype(x), Box<int>>);

```

Listing 17: Chain Composition (Bind / $\gg=$)

This mapping ensures that the "Categorical Cement" holding the "Algebraic Bricks" together is both syntactically expressive and computationally invisible.

4 Implementation: The Compile-Time Logic Engine

The core of `dedekind` is a recursive template evaluator that treats C++ `concepts` as logical predicates. Unlike traditional template metaprogramming that relies on SFINAE, we utilize C++23's `requires` expressions to perform "structural inspection" of types.

4.1 The Compiler as a Deterministic Proof Assistant

In the `dedekind` framework, the C++ compiler acts as a decidability engine for our mathematical ontology. Rather than relying on traditional class hierarchies, we use C++23 `Concepts` to define the structural requirements of a species. A type is not explicitly inherited from a base; instead, it is verified to be compliant with a species' `concept` through its intrinsic properties. This allows for a structuralist approach where any type—primitive or user-defined—can participate in the mereology if it satisfies the necessary algebraic invariants. This verification is supported by Template Specialization to define axiomatic base cases and SFINAE to prune invalid overloads. We leverage the Curiously Recurring Template Pattern (CRTP) within our internal wrappers to provide a unified interface for disparate types without the overhead of dynamic dispatch. By conducting these checks in a `constexpr` context and utilizing Move Semantics for resource management, we ensure that the "Proof" of a mathematical construction is resolved entirely at compile-time, leaving only optimized machine code.

```

// Verifying that the intersection of disjoint sets is empty
using Evens = dedekind::set<int, [](auto x){ return x % 2 == 0; }>;
using Odds = dedekind::set<int, [](auto x){ return x % 2 != 0; }>;
using Intersection = dedekind::meet_t<Evens, Odds>;
// The compiler resolves this to the Empty Set species
static_assert(dedekind::is_empty_v);
static_assert(sizeof(Intersection) == 1); // No machine state required

```

Listing 18: Compile-time inference of set invariants

```

// In dedekind, a valid program is a constructive proof.
// If the 'Proof' (the type-transformation) fails, the 'Program' does not compile.
template
requires dedekind::is_archimedean_field_v
auto compute_limit(T sequence) {
return sequence.limit();
}
// Attempting to compute a limit on a Discrete Ring:
// The compiler acts as a proof-assistant and rejects the invalid mathematical path
// Error: constraints not satisfied for 'is_archimedean_field_v'
auto result = compute_limit(my_discrete_ring);

```

Listing 19: The Reverse Curry-Howard Isomorphism in action

4.2 Reifying the Set-Builder

To achieve the syntax

$$\text{Set}\{x \% \mathbb{R} \wedge (x + 5 < 10)\} \quad (5)$$

, we introduce a `set` template that accepts a domain and a predicate. The predicate is not a function pointer, but a *Stateless Constraint Type*.

```

import dedekind.numbers;
import dedekind.logic;
// Define a predicate: x is even and greater than 10
auto P = [](auto x) {
return (x % 2 == 0) && (x > 10);
};
// Materialize the set at compile-time
using EvenGreaterTen = dedekind::set<int, P>;
static_assert(dedekind::contains(12));
static_assert(!dedekind::contains(7));

```

Listing 20: Example of Set-Builder realization in Dedekind

4.3 The Role of C++23 Modules

The stratification mentioned in Section 2 is strictly enforced via the module system. By using `export module dedekind.ontology:mereology`, we ensure that the compiler can cache the "structural truths" of the system. This prevents the exponential blow-up of compile times typically associated with heavy template libraries, as the "laws" of the ontology are compiled once into binary module interfaces (BMIs)

4.4 Continuous Integration

We utilize GitHub Actions to verify the project's structural integrity; each commit is checked via `static_assert` compile-time proofs and a suite of automated tests. Following the tradition of property-based testing [14], we utilize these tests as "runtime witnesses" to verify the materialization of our categorical primitives, aiming for a patch coverage threshold of 60% or higher for all new module partitions. While the automation of build and test cycles is considered a 'bread-and-butter' practice in contemporary software engineering [33], its application in the `dedekind` framework serves a specific epistemological purpose. By treating the CI pipeline as a continuous verification engine, we ensure that the categorical ontology remains decidable and physically materialized across all translation units.

5 Discussion

However, reifying a formal ontology in a statically-typed **systems** language presents significant challenges compared to "pure" functional languages like Haskell [34] or Agda [35], which have native support for higher-kinded types, dependent types and first-class functions. Challenges encountered include:

- **Template Template Parameters:** C++ is notoriously bad at true Category Theory because it lacks higher-kinded Types (HKTs). As a result `Functor<F>` or `Monad<F>` are defined in terms of `template template`, which have significant edge cases (e.g., they don't play well with aliases or multiple template arguments).
- **The Lambda Hurdle (Opaqueness):** Unlike the first-class, inspectable functions in the ML family, C++ lambdas are unique, anonymous types. They do not inherently expose their domain or codomain to the type system. A morphism's domain and codomain must be inferred at the call site or explicitly provided by the programmer. Although if used as *Non-Type Template Parameters* (NTTP), they can be "inspected" them via `concept` checks or by wrapping them in a type that carries its own signature metadata.
- **Decidability vs. Instantiation:** While dependent-type systems use proof-search, C++ relies on template instantiation. This introduces the risk of "Late Detection," where a mathematical violation is only discovered when a set is materialized, rather than when it is defined. This can be mitigated by forcing evaluations into `constexpr` and `consteval` contexts.
- **The Performance-Rigidity Tradeoff:** High-level abstractions in C++ often incur a "template tax" in compile times. We utilize C++23 **Module Partitions** to create a directed acyclic graph of truth, caching structural invariants to ensure that formal rigor does not collapse the build pipeline.
- **The Specialization Constraint:** Unlike the unified pattern matching of functional languages, C++ relies on Partial Template Specialization to resolve structural properties. This creates a "Rigidity Challenge": the compiler cannot readily deduce that a property of A also applies to $A \cup B$ without explicit boilerplate. We address this by using `concept ... requires` as logical compile-time predicate, thereby simulating the polymorphic reach of a true dependent-type system.
- **The Precedence-Associativity Constraint:** A significant friction point in embedding categorical notation within the C++ grammar is the language's fixed set of `operator` candidates and fixed precedence and associativity rules. While the `operator>=>` is used to model the Monadic Bind ($\gg$), the ISO C++ standard defines compound assignment operators (including $\gg=$, $\ll=$, $+=$, $*=$, etc.) with **right-to-left associativity**. Consequently, a standard monadic chain such as $m \gg f \gg g = (m \gg f) \gg g$ is parsed by the compiler as $m \gg= (f \gg= g)$. As this makes no sense in a monadic chain, explicit grouping, e.g., $(m \gg= f) \gg= g$ is required. As an alternative, we provide `operator>>` ("Highway Notation"), but with slightly different categorical semantics.

6 Conclusion

This paper has explored the feasibility of embedding formal mereology within the C++23 type system through the `dedekind` library. By transitioning from the traditional object-oriented paradigm of information hiding toward a model of structural transparency, we have illustrated how the compiler can be utilized as a rudimentary constraint satisfaction engine. This approach

allows for the high-level representation of mathematical species, such as set-builder notation, without the typical runtime performance penalties associated with abstract modeling.

Our investigation suggests that while the Clang/LLVM toolchain is not a dedicated theorem prover, it can effectively implement a ‘rethought’ set theory [36], where categorical axioms provide the semantic context for aggressive optimization. By hoisting mereological relations into the type signature, we enable the compiler to perform “structural pruning”—collapsing contradictions and optimizing intersections—resulting in machine code that maintains the efficiency of hand-tuned assembly.

The stratification of these formalisms into module partitions provides a scalable pathway for managing the “template tax,” ensuring that the rigors of formal logic do not destabilize the build pipeline. By embedding the Dedekind cut and the construction of the Continuum into the build process, we have moved toward a nascent paradigm of *Axiomatic Systems Programming*: a development style where the build process acts as a physical guardrail for mathematical truth.

Future work will investigate extending this mereological framework to other domains of the computational sciences and high-performance computing, such as a direct integration with the upcoming linear algebra capabilities of the C++ standard library [37] while expressly facilitating computations not just using floating point numbers but more generally supporting simplified algebraic systems such as semimodules over rings $(\mathbb{B}, \mathbb{N})$ or extended number systems such as the complex numbers $(\mathbb{C})$, the dual numbers or quaternions.

References

- [1] Adam Paszke, Sam Gross, Francisco Massa, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems 32*. Ed. by H. Wallach et al. 2019, pp. 8024–8035. URL: <https://neurips.cc>.
- [2] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.
- [3] Wenzel Jakob. *nanobind: tiny and efficient C++/Python bindings*. <https://github.com/wjakob/nanobind>. 2022.
- [4] ISO/IEC JTC 1/SC 22. *ISO/IEC 14882:2024: Programming languages — C++*. Commonly referred to as C++23. International Organization for Standardization, Geneva, Switzerland, 2024. URL: <https://www.iso.org>.
- [5] Ivan Čukić. *Functional Programming in C++*. Manning Publications, 2018. ISBN: 9781617293818.
- [6] Eugenio Moggi. “Notions of computation and monads”. In: *Information and computation* 93.1 (1991), pp. 55–92.
- [7] Steve Awodey. “An Answer to Hellman’s Question: Does Category Theory Provide a Framework for Mathematical Structuralism?” In: *Philosophia Mathematica* 12.1 (2004), pp. 54–64.
- [8] André Joyal. “Une théorie combinatoire des séries formelles”. In: *Advances in Mathematics* 42.1 (1981), pp. 1–82. DOI: 10.1016/0001-8708(81)90052-9.
- [9] André Joyal. “Fonctions analytiques et espèces de structures”. In: *Combinatoire énumérative*. Springer, 1986, pp. 126–159. DOI: 10.1007/BFb0072514.
- [10] Colin McLarty. “Numbers can be just what they have to”. In: *Noûs* 27.4 (1993), pp. 487–498.
- [11] John Baez, Urs Schreiber, and David Corfield. *The n-Category Café: A group blog on math, physics and philosophy*. <https://utexas.edu>. 2006–present.

- [12] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Provides the formal specification for floating-point formats and operations. 2019. DOI: 10.1109/IEEESTD.2019.8766229.
- [13] Martin Thompson. *Mechanical Sympathy*. Mechanical Sympathy Blog. 2011. URL: <https://mechanical-sympathy.blogspot.com/2011/07/why-mechanical-sympathy.html> (visited on 03/31/2026).
- [14] Koen Claessen and John Hughes. “QuickCheck: a lightweight tool for random testing of Haskell programs”. In: *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 2000, pp. 268–279.
- [15] Bryan O’Sullivan, John Goerzen, and Don Stewart. *Real World Haskell*. O’Reilly Media, 2008.
- [16] Philip Wadler. “Theorems for free!” In: *Conference on Functional Programming Languages and Computer Architecture*. ACM. 1989, pp. 347–359.
- [17] Bartosz Milewski. *Category Theory for Programmers*. Compiled from the original blog series at <https://bartoszmilewski.com>. Blurb, Incorporated, 2019. ISBN: 9780464243878.
- [18] Kazimierz Ajdukiewicz. *Język i Poznananie (Language and Cognition)*. Foundational text for the Polish School of Logic and Semantic Epistemology. Państwowe Wydawnictwo Naukowe, 1985.
- [19] Colin McLarty. *Elementary Categories, Elementary Toposes*. Clarendon Press, 1992.
- [20] F. William Lawvere. “An elementary theory of the category of sets”. In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.
- [21] F William Lawvere. “An elementary theory of the category of sets (long version) with commentary”. In: *Reprints in Theory and Applications of Categories* 11 (2005), pp. 1–35.
- [22] Vincent Krätler. *dedekind: A Structuralist Mereology in C++23*. Version v1.0.0. 2026. URL: <https://github.com>.
- [23] Heinrich Kleisli. “Every standard construction is induced by a pair of adjoint functors”. In: *Proceedings of the American Mathematical Society* 16.3 (1965), pp. 544–546. DOI: 10.2307/2034658.
- [24] Simon Peyton Jones. *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press, 2003.
- [25] nLab authors. *Kleisli category*. Accessed: 2026-04-06. 2023.
- [26] Jean-Pierre Marquis. “The Structuralist Mathematical Style: Bourbaki as a Case Study”. In: *The Architecture of Modern Mathematics*. Oxford University Press, 2022.
- [27] F. William Lawvere and Robert Rosebrugh. *Sets for Mathematics*. Cambridge University Press, 2003.
- [28] F William Lawvere. “Metric spaces, generalized logic, and closed categories”. In: *Rendiconti del seminario matematico e fisico di Milano* 43.1 (1973), pp. 135–166.
- [29] Francis Borceux and Françoise Dejean. “Cauchy completion in category theory”. In: *Cahiers de topologie et géométrie différentielle catégoriques* 27.2 (1986), pp. 133–146.
- [30] Thomas Mormann. “Structural Mereology and Statistics”. In: *Logic and Logical Philosophy* 22.4 (2013), pp. 427–453.
- [31] Joachim Lambek and Philip J Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge University Press, 1988.
- [32] nLab authors. *structural set theory*. ncatlab.org/nlab/show/structural+set+theory. 2024. URL: <https://ncatlab.org>.

- [33] Paul M. Duvall, Steve Matyas, and Andrew Glover. *Continuous Integration: Improving Software Quality and Reducing Risk*. Signature Series. Addison-Wesley Professional, 2007.
- [34] The GHC Development Team. *The Glasgow Haskell Compiler User’s Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html.
- [35] Ulf Norell. “Towards a practical programming language based on dependent type theory”. PhD thesis. Chalmers University of Technology, 2007. URL: <http://www.cse.chalmers.se>.
- [36] Tom Leinster. “Rethinking set theory”. In: *The American Mathematical Monthly* 121.5 (2014). arXiv:1212.6543, pp. 403–415.
- [37] Mark Hoemmen et al. *P1673R13: A free function linear algebra interface based on the BLAS*. Tech. rep. ISO/IEC JTC1/SC22/WG21, 2024. URL: <https://wg21.link>.